Arquitectura del Software y Decisiones Técnicas

Proyecto final – AsistenteIA

Información	Descripción
Universidad	Universidad de San Buenaventura
Programa	Tecnología de Desarrollo de Software – VI semestre
Asignatura	Programación Avanzada y Aplicada
Docente	Sandra Esperanza Sánchez Barragán
Proyecto	AsistenteIA – Asistente académico móvil con IA (.NET MAUI, API REST, SQLite)
Integrantes	Juan Sebastián Garzón Suárez y Nicolás Quiroz

1. Estilo arquitectónico

La aplicación usa una **arquitectura en capas con el patrón MVVM** (Model–View–ViewModel) sobre .NET MAUI. Las dependencias se registran y se inyectan desde `MauiProgram.cs`. Cada capa tiene una responsabilidad única, lo que facilita el mantenimiento y la sustentación del código.

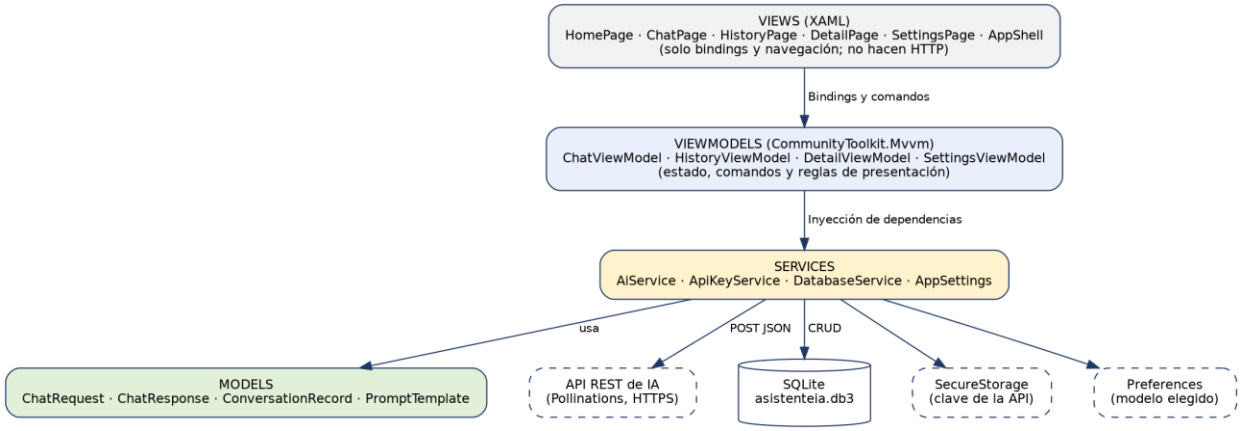


Figura 1. Arquitectura en capas de AsistenteIA.

2. Capas y responsabilidades

Capa	Carpeta	Responsabilidad	Componentes
Presentación (View)	Views	Interfaz XAML y navegación. Solo enlaza propiedades y comandos; no realiza solicitudes HTTP.	HomePage, ChatPage, HistoryPage, DetailPage, SettingsPage, AppShell
Lógica de presentación (ViewModel)	ViewModels	Estado, comandos y reglas de presentación.	ChatViewModel, HistoryViewModel, DetailViewModel, SettingsViewModel

Capa	Carpeta	Responsabilidad	Componentes
Servicios	Services	Acceso a la API, la base de datos, la conectividad y el almacenamiento seguro.	AiService, DatabaseService, ApiKeyService, AppSettings
Modelos	Models	Representan el JSON de la API y las entidades locales.	ChatRequest, ChatMessage, ChatResponse, ChatChoice, ConversationRecord, PromptTemplate
Plataforma	—	Servicios nativos de .NET MAUI usados por los servicios.	SecureStorage, Preferences, Connectivity, HttpClient, SQLite

3. Flujo entre capas

XAML (ChatPage) → **ChatViewModel** (AskCommand) → **AiService** (POST) → **API REST de IA** (JSON) → **ChatViewModel** (Answer) → **DatabaseService** (SaveAsync) → **SQLite** (asistenteia.db3).

4. Inyección de dependencias

Ciclo de vida	Componentes	Motivo
Singleton	HttpClient, ApiKeyService, AiService, DatabaseService	Un solo HttpClient reutilizable y una sola conexión a la base de datos.
Transient	ViewModels y Pages	Estado limpio cada vez que se crea una pantalla.

5. Navegación

Se usa **Shell** con una `TabBar` de cuatro pestañas (Inicio, Consultar, Historial y Configuración). La pantalla **Detalle** se registra como ruta con `Routing.RegisterRoute` y recibe el identificador del registro por parámetro (`DetailPage?id=`), que el ViewModel recibe con `QueryProperty`.

6. Decisiones técnicas

Decisión	Alternativa descartada	Justificación
MVVM con CommunityToolkit.Mvvm	Código en el code-behind	Separa responsabilidades y reduce código repetido con [ObservableProperty] y [RelayCommand].
Shell con TabBar y ruta para Detalle	NavigationPage manual	Navegación declarativa y rutas con parámetros.
SQLite (sqlite-net-pcl)	Archivos JSON	Permite consultas, filtros y CRUD con persistencia verificable.
SecureStorage para la clave	Constante en el código o archivo de configuración	Evita exponer credenciales; usa el almacén seguro de la plataforma.
HttpClient singleton con BaseAddress	Un cliente por cada llamada	Reutiliza conexiones y evita el agotamiento de sockets.

Decisión	Alternativa descartada	Justificación
Modelo y endpoint configurables	Modelo fijo en el código	El proveedor puede cambiar modelos, cuotas o autenticación.
Clave ingresada por el usuario	Clave incluida en la aplicación	Una aplicación móvil no puede guardar secretos; solo se usan credenciales publicables.
CancellationToken y tiempo máximo de 60 s	Llamadas sin cancelación	La interfaz no se bloquea y el usuario puede reintentar.
Validación del prompt en el servicio (5 a 2000 caracteres)	Validar solo en la interfaz	La regla se cumple aunque cambie la vista.
Destino Windows y Android	iOS / macOS	No requiere un equipo Mac y cumple el RF10 (Windows o Android).